

Keyboard Shortcuts

Max Burchell

Abstract

The Symbol Entry System is an embedded hardware and software platform designed to simplify the entry of special symbols into desktop applications. The system uses an NXP FRDM-MCXN947 microcontroller with physical button inputs to navigate and select predefined symbols including π , Σ , μ , Ω , and $\int$ (it's designed to be easy to add or remove whatever symbols are desired; these are just the base symbols). A companion Python application communicates with the microcontroller through USB serial communication and automatically inserts the selected symbol into the active desktop application using cross-platform keyboard emulation techniques. The project integrates embedded firmware, serial communication, operating-system-specific input handling, and real-time user interface design. Timing analysis demonstrated menu update response times averaging approximately 362 μ s, significantly below the required 50 ms system limit. The project demonstrates concepts in embedded systems, hardware/software integration, and low-latency system design while providing a practical productivity tool for engineering and scientific applications.

Background

Technical fields such as engineering, physics, and mathematics rely heavily on specialized symbols— π , Ω , $\int$, μ , and Σ —that are not easily accessible on standard keyboards. Students, researchers, and professionals often resort to slow and inconsistent input methods, including navigating symbol menus, memorizing platform-specific key combinations, or copying symbols from external sources. These interruptions disrupt workflow and accessibility, particularly for users who rely on consistent and rapid mathematical notation entry.

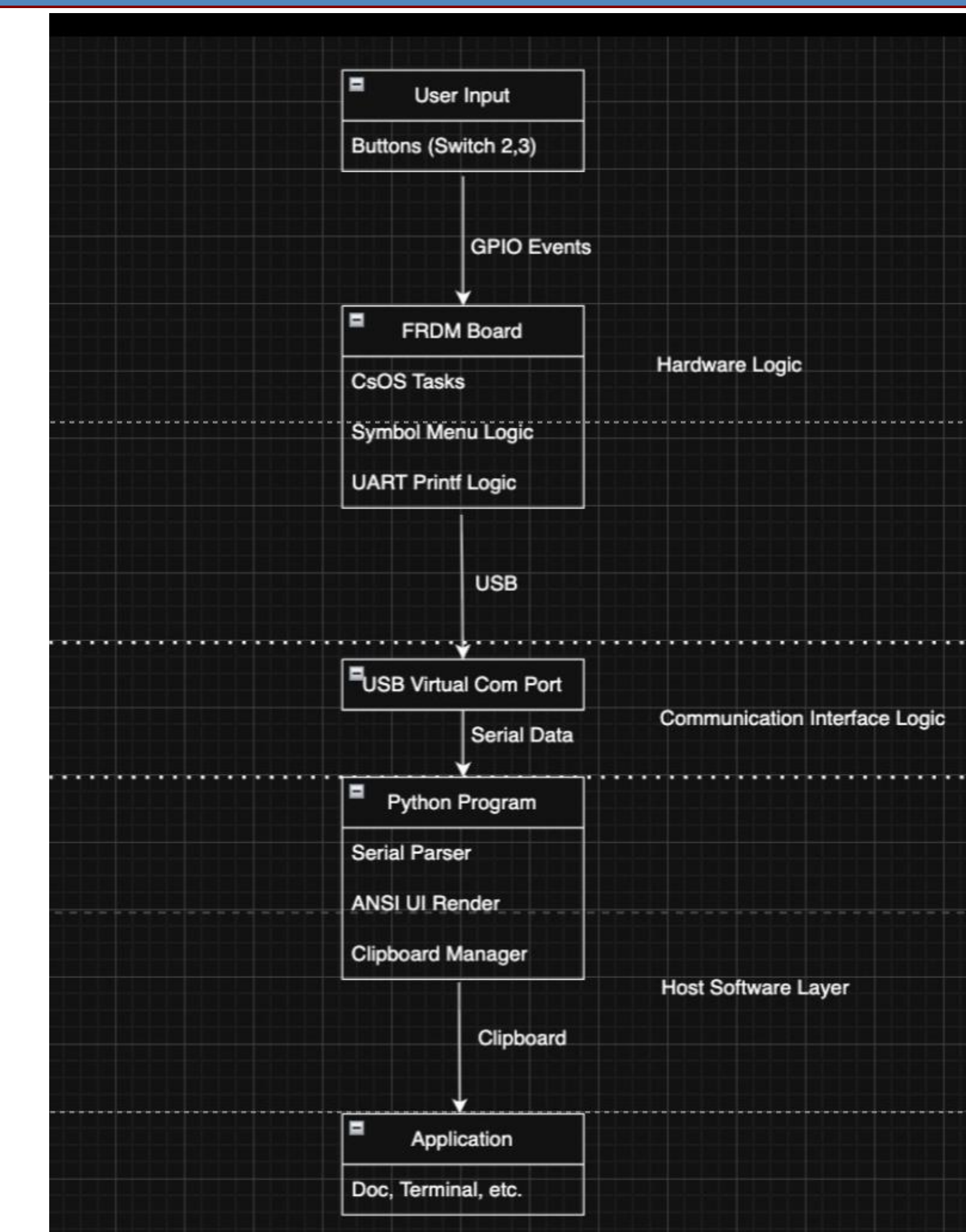
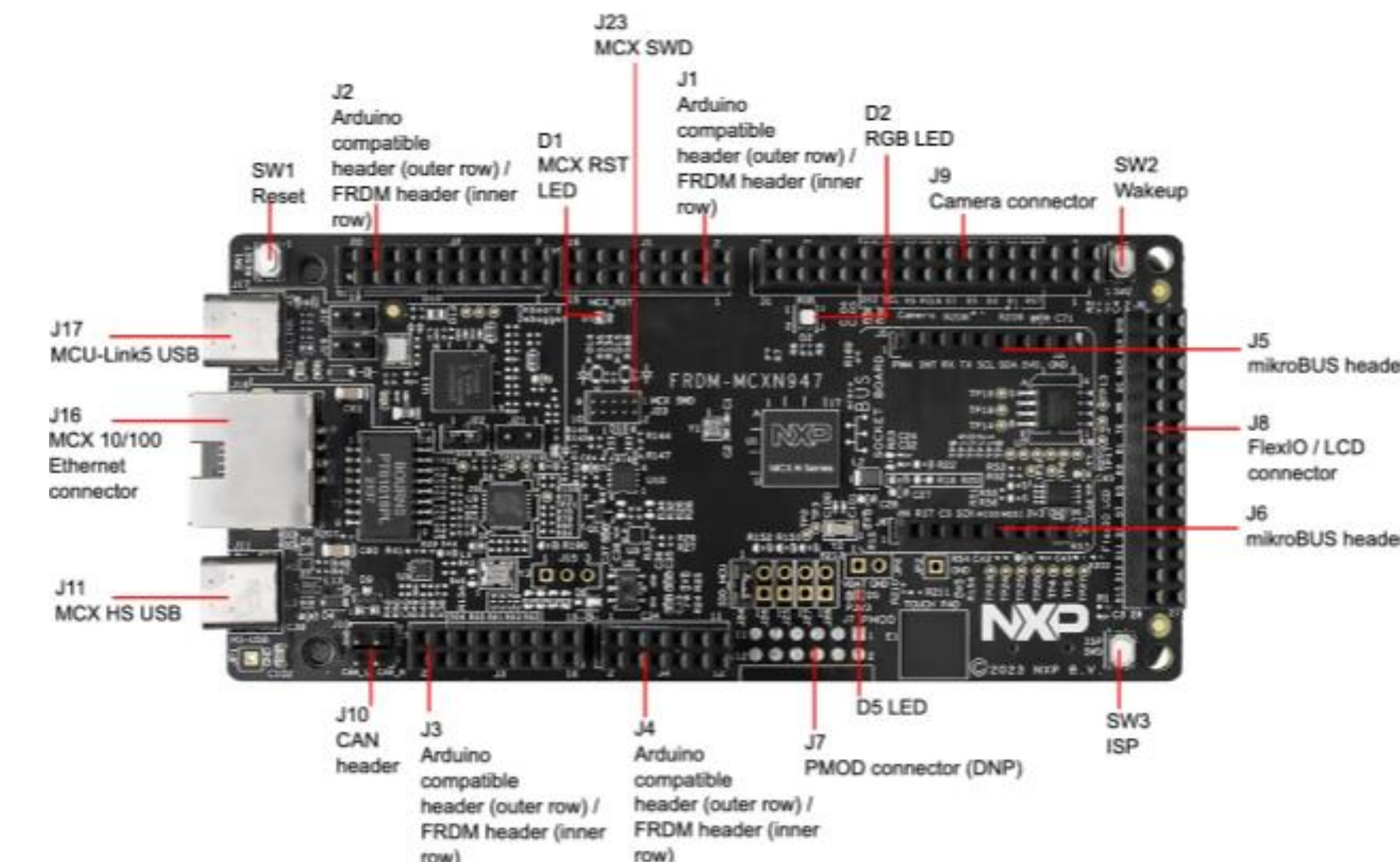
To address this problem, this project develops a dedicated hardware input device that enables quick and intuitive entry of commonly used symbols. The Symbol Entry System targets a broad audience including STEM students, educators, and engineers who produce technical documents on a daily basis. By allowing users to cycle through a symbol list and send the currently selected character directly into their active document, the system reduces friction and improves efficiency in technical communication.

References

[Firmware](#) (Open-Source Code for keyboard programming)

Methods and Materials

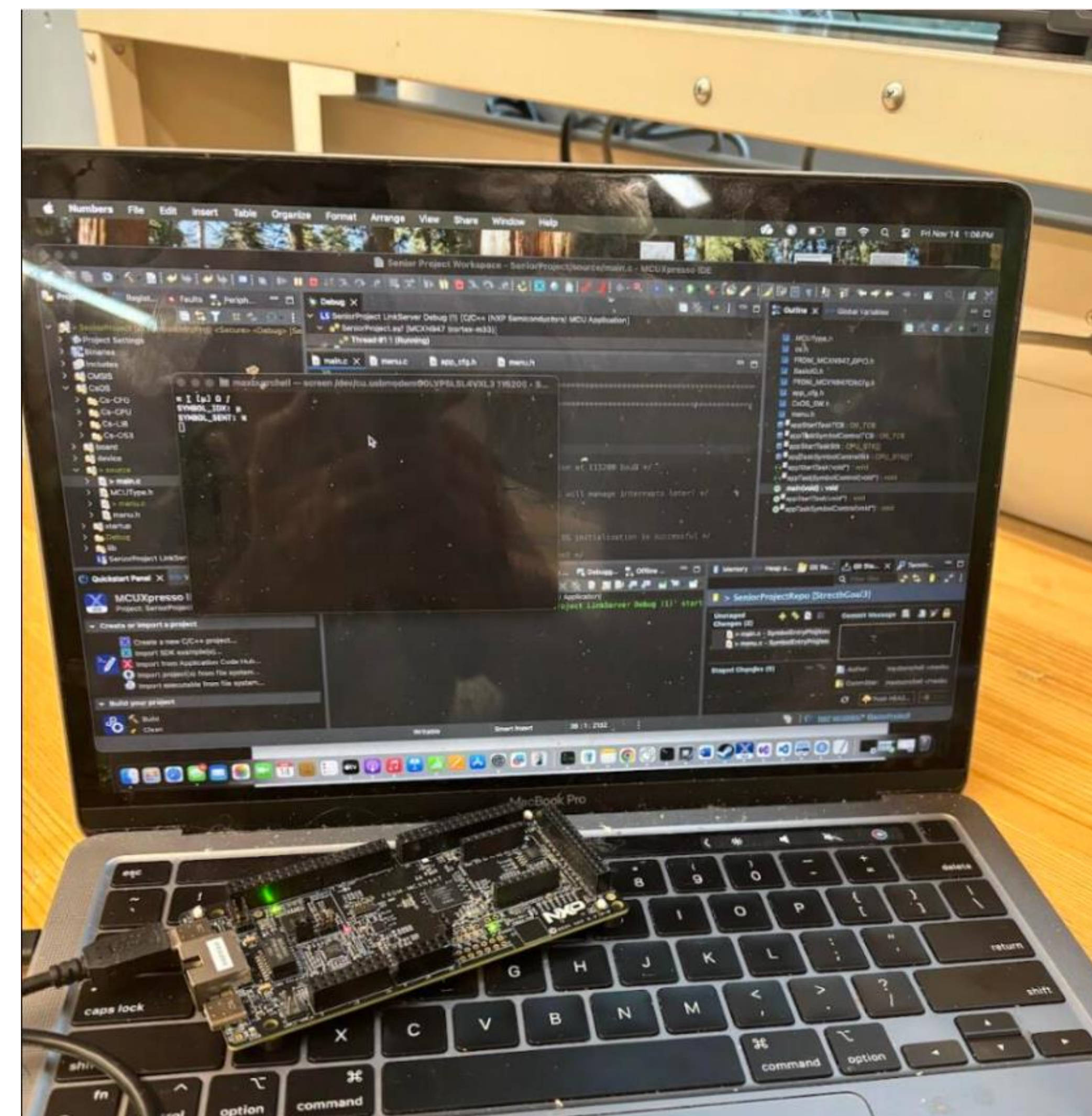
- The MCU stores and cycles through predefined symbols using onboard pushbuttons.
- The selected symbol is transmitted to the host computer through USB serial communication.
- The Python host application interprets the incoming serial messages.
- Platform-specific keyboard emulation functions insert the selected symbol into the active application.
- ANSI terminal rendering is used to provide a live updating menu interface with minimal latency.



Results

- Average menu update latency: 361.9 μ s
- Standard deviation: 16.38 μ s
- Successfully operated on both Windows and macOS
- Maintained stable USB communication within the required temperature range of 0 $^{\circ}$ C to 40 $^{\circ}$ C
- Total hardware cost remained below the \$50 project requirement (\$26.95 with free shipping)

A statistical analysis showed that the measured timing performance remained several orders of magnitude below the required 50 ms latency limit.



Future Direction

- Native USB HID keyboard implementation (In progress)
- Expanded symbol and macro libraries
- Custom graphical desktop application
- Wireless connectivity support

Acknowledgements

Special thanks to Todd Morton for providing base SDK files for MCU